

Undergraduate Topics in Computer Science

Maurizio Gabbrielli
Simone Martini

Programming Languages: Principles and Paradigms

Second Edition



 Springer

Undergraduate Topics in Computer Science

Series Editor

Ian Mackie, University of Sussex, Brighton, UK

Advisory Editors

Samson Abramsky , Department of Computer Science, University of Oxford, Oxford, UK

Chris Hankin , Department of Computing, Imperial College London, London, UK

Mike Hinchey , Lero – The Irish Software Research Centre, University of Limerick, Limerick, Ireland

Dexter C. Kozen, Department of Computer Science, Cornell University, Ithaca, NY, USA

Andrew Pitts , Department of Computer Science and Technology, University of Cambridge, Cambridge, UK

Hanne Riis Nielson , Department of Applied Mathematics and Computer Science, Technical University of Denmark, Kongens Lyngby, Denmark

Steven S. Skiena, Department of Computer Science, Stony Brook University, Stony Brook, NY, USA

Iain Stewart , Department of Computer Science, Durham University, Durham, UK

Joseph Migga Kizza, College of Engineering and Computer Science, The University of Tennessee-Chattanooga, Chattanooga, TN, USA

‘Undergraduate Topics in Computer Science’ (UTiCS) delivers high-quality instructional content for undergraduates studying in all areas of computing and information science. From core foundational and theoretical material to final-year topics and applications, UTiCS books take a fresh, concise, and modern approach and are ideal for self-study or for a one- or two-semester course. The texts are all authored by established experts in their fields, reviewed by an international advisory board, and contain numerous examples and problems, many of which include fully worked solutions.

The UTiCS concept relies on high-quality, concise books in softback format, and generally a maximum of 275–300 pages. For undergraduate textbooks that are likely to be longer, more expository, Springer continues to offer the highly regarded Texts in Computer Science series, to which we refer potential authors.

Maurizio Gabbrielli · Simone Martini

Programming Languages: Principles and Paradigms

Second Edition

With a chapter by Saverio Giallorenzo

Maurizio Gabbrielli
Department of Computer Science
and Engineering
University of Bologna
Bologna, Italy

Simone Martini
Department of Computer Science
and Engineering
University of Bologna
Bologna, Italy

ISSN 1863-7310

ISSN 2197-1781 (electronic)

Undergraduate Topics in Computer Science

ISBN 978-3-031-34143-4

ISBN 978-3-031-34144-1 (eBook)

<https://doi.org/10.1007/978-3-031-34144-1>

1st edition: © Springer-Verlag London Ltd., part of Springer Nature 2010

2nd edition: © The Editor(s) (if applicable) and The Author(s), under exclusive license to Springer
Nature Switzerland AG 2023

This work is subject to copyright. All rights are solely and exclusively licensed by the Publisher, whether the whole or part of the material is concerned, specifically the rights of reprinting, reuse of illustrations, recitation, broadcasting, reproduction on microfilms or in any other physical way, and transmission or information storage and retrieval, electronic adaptation, computer software, or by similar or dissimilar methodology now known or hereafter developed.

The use of general descriptive names, registered names, trademarks, service marks, etc. in this publication does not imply, even in the absence of a specific statement, that such names are exempt from the relevant protective laws and regulations and therefore free for general use.

The publisher, the authors, and the editors are safe to assume that the advice and information in this book are believed to be true and accurate at the date of publication. Neither the publisher nor the authors or the editors give a warranty, expressed or implied, with respect to the material contained herein or for any errors or omissions that may have been made. The publisher remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

This Springer imprint is published by the registered company Springer Nature Switzerland AG
The registered company address is: Gewerbestrasse 11, 6330 Cham, Switzerland

*To Francesca and Antonella,
who will never want to read this book
but who contributed to it being written.*

*To Costanza, Maria and Teresa, who will
read it perhaps,
but who have done everything to stop it being
written.*

*In the end not only Francesca and Antonella
never read the book,
but also Costanza, Maria, and Teresa never
got to do it.
This second edition is dedicated also to
Daniela,
hoping that at least she will read it.*

Foreword to the First Edition

With great pleasure, I accepted the invitation extended to me to write these few lines of Foreword. I accepted for at least two reasons. The first is that the request came to me from two colleagues for whom I have always had the greatest regard, starting from the time when I first knew and appreciated them as students and as young researchers.

The second reason is that the text by Gabbrielli and Martini is very near to the book that I would have liked to have written but, for various reasons, never have. In particular, the approach adopted in this book is the one which I myself have followed when organizing the various courses on programming languages I have taught for almost 30 years at different levels under various titles.

The approach, summarized in two words, is that of introducing the general concepts (either using linguistic mechanisms or the implementation structures corresponding to them) in a manner that is independent of any specific language; once this is done, “real languages” are introduced. This is the only approach that allows one to reveal similarities between apparently quite different languages (and also between paradigms). At the same time, it makes the task of learning different languages easier. In my experience as a lecturer, ex-students recall the principles learned in the course even after many years; they still appreciate the approach which allowed them to adapt to technological developments without too much difficulty.

The book by Gabbrielli and Martini has, as central reference point, an undergraduate course in Computer Science. For this reason, it does not have complex prerequisites and tackles the subject by finding a perfect balance between rigor and simplicity. Particularly appreciated and successful is the force with which they illuminate the connections with other important “areas of theory” (such as formal languages, computability, semantics) which the book rightly includes (a further justification for their inclusion being that these topics are no longer taught in many degree courses).

Pisa, Italy
Fall 2005

Giorgio Levi

Preface to the Second Edition

For this second English edition, we have extensively revised the translation of the first printing. As well as correcting errata, we have updated the existing material, clarified some tricky points, and discussed newer programming languages, with Python as the main reference. Almost every page of the previous edition has changed in some way.

We also added three final chapters, on topics that we felt are too important today to be omitted, even in an undergraduate level textbook.

The first regards constraint programming. It represents the third approach seen in this book to declarative programming, which has a wide range of applications in artificial intelligence. We discuss the main ideas behind the paradigm and we illustrate the two main ways to implement it: constraint logic programs and constraint programming in the context of an imperative language. For the latter approach, we use MinZinc as a reference language. Following the guiding principle of the book, we do not want to teach constraint logic programming, MinZinc or any other specific language, but we rather provide enough background for appreciating the potential that constraint programming has in many application areas.

The second new chapter concerns concurrent programming. Although our aim remains that of an introductory textbook, the absence of any reference to concurrency seemed to us too conspicuous a gap, given that a significant part of software today exploits concurrency, from the operating system level up to the Web services. An exhaustive treatment of this topic would require (at least) a volume by itself. The chapter illustrates only the main problems that arise when switching from sequential to concurrent programs, together with the relative solutions. We tried to offer an adequate panorama of the main techniques and linguistic constructs for realizing interaction mechanisms, synchronization, and communication between concurrent programs or processes. Following the guiding principle of the whole book, we have not referred to a specific language, even if we have tried to make some notions concrete by examining the case of Java.

The third new chapter, written by Saverio Giallorenzo, in a way, rounds up the tour we started in the chapter on concurrency. We focus on a specific form of the concurrent paradigm, distributed programming, and we introduce service orientation. Distribution (as in “processes not sharing memory”) presents both challenges and opportunities. We highlight how dedicated programming language constructs

have evolved to tackle these issues and seize their strengths to support the development of open-ended, complex distributed programs through principles such as loose coupling and interoperability. We also explore the fundamental concepts of services, their traits, the related language constructs, and the challenges and support for verifying the correctness of service-oriented programs. As done in the previous chapters, we never focus on a specific language, and we maintain an open perspective. We explore how languages from this landscape implemented the same concepts, introduced different solutions to common issues, or rather evolved following different interpretations of the same ideas. In the chapter, we also provide an overview of the role of service-oriented architectures and cloud computing in the development and evolution of the service-oriented paradigm.

Use of the Book

The book is primarily a university textbook, but it is also suitable for the personal education of IT specialists who want to deepen their knowledge of the mechanisms behind the languages they use. The choice of themes and the presentation style are largely influenced by the experience of teaching the content as part of the bachelor's degree in Computer Science of the School of Sciences at *Alma Mater Studiorum*—Università di Bologna.

In our experience, the book can cover two modules of six credits, placed in the second or third year of the bachelor's degree. A first module can cover abstract machines and the fundamental aspects, say Chaps. 1–9. The other can cover (some of) the different paradigms. As the maturity of students increases it also increases the amount of material one can present.

Acknowledgements

M. G. and S. M. are happily indebted to Saverio Giallorenzo for his prodding to finish this second edition. Besides writing Chap. 15 on service-oriented programming, he reviewed the chapter on concurrency, revised and extended the one on history, and made many more valuable contributions to the text of the whole book.

We would like to thank the people who have kindly helped us with the new chapters included in the second edition. Roberto Amadini read a draft of the chapter on constraint programming and provided useful insights and comments. Fabrizio Montesi and Florian Rademacher read drafts of the chapter on service orientation and helped to improve it with their generous feedback, insights, and errata.

Bologna, Italy
Spring 2023

Maurizio Gabbrielli
Simone Martini
Saverio Giallorenzo

Preface to the First Edition

Facilius per partes in cognitionem totius adducimur
(Seneca, *Epist.* 89, 1)

Learning a programming language, for most students in computing, is akin to a rite of passage. It is an important transition, soon recognized as insufficient. Among the tools of the trade, there are many languages, so an important skill for a good IT specialist is to know how to move from one language to another (and how to learn new ones) with naturalness and speed.

This competence is not obtained by learning many different languages from scratch. Programming languages, like natural languages, have their similarities, analogies, cross-pollination phenomena, and genealogies that influence their characteristics. If it is impossible to learn tens of languages well, it is possible to deeply understand the mechanisms that inspire and guide the design and implementation of hundreds of different languages. This knowledge of the “parts” eases the understanding of the “whole” of a new language and, therefore, underpins a fundamental methodological competence in the professional life of IT specialists, at least as far as it allows them to anticipate innovations and outlive technologies that grow obsolete.

It is for these reasons that a course on the general aspects of programming languages is, throughout the world, a key step in the advanced education (university or professional) of IT specialists. The fundamental competences which an IT specialist must possess about programming languages are of at least four types:

- the proper linguistic aspects;
- how one can implement language constructs and what are the costs relative to a given implementation;
- the architectural aspects that influence implementation;
- translation techniques (compilation).

A single course rarely deals with all four of these aspects. In particular, description of the architectural aspects and compilation techniques are both topics that are sufficiently complex and elaborate to deserve independent courses. The remaining

aspects are primarily the content of a general course on programming languages and comprise the main subject of this book.

The literature is rich in texts dealing with these subjects and generations of students have used them in their education. All these texts, though, have in mind an advanced reader who already understands many programming languages, who has a more-than-superficial competence with the fundamental basic mechanisms, and who is not afraid when confronted with a fragment of code written in an unknown language (because they can understand it by looking at the analogies and differences with those they already know). We can see these textbooks as focused on the comparison among languages. These are long, deep, and stimulating books, but they are *too* long and deep (read: difficult) for the student who begins their career with a single programming language (or at most two) and who still has to learn the basic concepts in detail.

This book aims to fill this gap. Experts will see that the index of topics pursues in large measure classical themes. However, we treat these themes in an elementary fashion, trying to assume as prerequisites only the bare minimum. We also strive to not make the book a catalogue of the traits and differences of the existing programming languages. The ideal (or reference) reader knows one language (well) (for example, Pascal, C, C++, or Java); even better if they have had some exposure to another language or paradigm. We avoided references to languages that are now obsolete, and we rarely show code examples written in a specific programming language. Mainly, the text freely uses a sort of pseudo-language (whose concrete syntax was inspired by C and Java) and seeks, in this way, to describe the most relevant aspects of different languages.

Every so often, a “box” at the top of the page presents an in-depth discussion, a reminder of a basic concept or specific details about popular languages (C, C++, Java; ML, and LISP for functional languages; PROLOG for logic programming languages). The reader at the first reading can safely skip the material in boxes.

Every chapter contains a short sequence of exercises, intended as a way of testing the understanding of the material. There are no truly difficult exercises or requiring more than 10 min for their solution.

Chapter 3 (Foundations) deals with themes that are not usually present in a book on programming languages. It is natural, however, while discussing static semantics and comparing languages, to ask what are the limits of the static analysis of programs and whether one can apply the same techniques to different languages. Rather than pointing the reader to other textbooks, given the conceptual and pragmatic relevance of these questions, we decided to answer them directly. In the space of a few pages, we present, in an informal but rigorous manner, the undecidability of the halting problem and that all general-purpose programming languages express the same class of computable functions. This allows us to present to the student, who does not always have a complete course on the “fundamentals” in their curriculum, the main results related to the limitations of computational models, which we deem fundamental for their education.

Besides the principles, this book also introduces the main *programming paradigms*: object-oriented, functional and logic. The need to write an introductory

book and keep it within a reasonable length explains the exclusion of important topics, such as concurrency and scripting languages for example.

Acknowledgements

Our thanks to Giorgio Levi go beyond the fact that he had the grace to write the Foreword. Both of us owe to him our first understanding of the mechanisms that underpin programming languages. His teaching appears in this book in a way that is anything but marginal.

We have to thank the many people who kindly helped us in the realization of the book. Ugo Dal Lago drew the figures using METAPOST and Cinzia Di Giusto, Wilmer Ricciotti, Francesco Spegni, and Paolo Tacchella read and commented on the drafts of some chapters. The following people pointed out misprints and errors: Irene Borra, Ferdinanda Camporesi, Marco Comini, Michele Filannino, Matteo Friscini, Stefano Gardenghi, Guido Guizzunti, Giacomo Magisano, Flavio Marchi, Fabrizio Massei, Jacopo Mauro, Maurizio Molle, Mirko Orlandelli, Marco Pedicini, Andrea Rappini, Andrea Regoli, Fabiano Ridolfi, Giovanni Rosignoli, Giampiero Travaglini, and Fabrizio Giuseppe Ventola.

We gladly acknowledge the support of the Department of Computer Science of Università di Bologna towards the English translation of the first edition of the book.

Bologna, Italy
Fall 2005

Maurizio Gabbrielli
Simone Martini

Contents

1	Abstract Machines	1
1.1	The Concepts of Abstract Machine and the Interpreter	1
1.1.1	The Interpreter	2
1.1.2	An Example of an Abstract Machine: The Hardware Machine	5
1.2	Implementation of a Language	9
1.2.1	Implementation of an Abstract Machine	9
1.2.2	Implementation: The Ideal Case	11
1.2.3	Implementation: The Real Case and the Intermediate Machine	16
1.3	Hierarchies of Abstract Machines	20
1.4	Summary	22
1.5	Bibliographical Notes	23
1.6	Exercises	23
	References	24
2	Describing a Programming Language	25
2.1	Levels of Description	25
2.2	Grammar and Syntax	27
2.2.1	Context-Free Grammars	28
2.3	Contextual Syntactic Constraints	37
2.4	Compilers	39
2.5	Semantics	43
2.6	Pragmatics	50
2.7	Implementation	50
2.8	Summary	50
2.9	Bibliographical Notes	51
2.10	Exercises	51
	References	52
3	Foundations	53
3.1	The Halting Problem	53
3.2	Undecidable Problems	55
3.2.1	The Standard Model	56
3.2.2	More Undecidable Problems	57

3.3	Formalisms for Computability	57
3.4	There Are More Functions Than Algorithms	59
3.5	Summary	60
3.6	Bibliographical Notes	61
3.7	Exercises	61
	References	62
4	Names and the Environment	63
4.1	Names and Denotable Objects	63
4.1.1	Denotable Objects	65
4.2	Environments and Blocks	66
4.2.1	Blocks	67
4.2.2	Types of Environment	68
4.2.3	Operations on Environments	71
4.3	Scope Rules	73
4.3.1	Static Scope	74
4.3.2	Dynamic Scope	76
4.3.3	Some Scope Problems	78
4.4	Summary	80
4.5	Bibliographical Notes	81
4.6	Exercises	81
	References	85
5	Memory Management	87
5.1	Techniques for Memory Management	87
5.2	Static Memory Management	89
5.3	Dynamic Memory Management Using Stacks	90
5.3.1	Activation Records for In-Line Blocks	92
5.3.2	Activation Records for Procedures	93
5.3.3	Stack Management	96
5.4	Dynamic Management Using a Heap	97
5.4.1	Fixed-Length Blocks	98
5.4.2	Variable-Length Blocks	99
5.5	Implementation of Scope Rules	102
5.5.1	Static Scope: The Static Chain	102
5.5.2	Static Scope: The Display	107
5.5.3	Dynamic Scope: Association Lists and CRT	109
5.6	Summary	113
5.7	Bibliographical Notes	114
5.8	Exercises	114
	References	116
6	Control Structure	117
6.1	Expressions	118
6.1.1	Expression Syntax	118
6.1.2	Semantics of Expressions	120
6.1.3	Evaluation of Expressions	123

6.2	The Concept of Command	127
6.2.1	The Variable	128
6.2.2	Assignment	129
6.3	Sequence Control Commands	134
6.3.1	Commands for Explicit Sequence Control	135
6.3.2	Conditional Commands	137
6.3.3	Iterative Commands	141
6.4	Structured Programming	148
6.5	Recursion	151
6.5.1	Tail Recursion	153
6.5.2	Recursion or Iteration?	157
6.6	Summary	158
6.7	Bibliographical Notes	159
6.8	Exercises	159
	References	161
7	Control Abstraction	163
7.1	Subprograms	164
7.1.1	Functional Abstraction	166
7.1.2	Parameter Passing	167
7.2	Higher-Order Functions	176
7.2.1	Functions as Parameters	177
7.2.2	Functions as Results	182
7.2.3	Anonymous Functions: Lambda Expressions	185
7.3	Exceptions	187
7.3.1	Implementing Exceptions	192
7.4	Summary	193
7.5	Bibliographical Notes	194
7.6	Exercises	195
	References	197
8	Structuring Data	199
8.1	Data Types	199
8.1.1	Types as Support for Conceptual Organisation	200
8.1.2	Types for Correctness	201
8.1.3	Types and Implementation	202
8.2	Type Systems	203
8.2.1	Static and Dynamic Checking	204
8.3	Scalar Types	205
8.3.1	Booleans	206
8.3.2	Characters	207
8.3.3	Integers	207
8.3.4	Reals	207
8.3.5	Fixed Point	208
8.3.6	Complex	209
8.3.7	Unit	209

8.3.8	Enumerations	210
8.3.9	Intervals	210
8.3.10	Ordered Types	211
8.4	Composite Types	211
8.4.1	Records	211
8.4.2	Unions	213
8.4.3	Tagged Unions	215
8.4.4	Arrays	216
8.4.5	Sets	221
8.4.6	Pointers	222
8.4.7	Sequences	226
8.4.8	Recursive Types	228
8.4.9	Functions	230
8.5	Equivalence	231
8.5.1	Equivalence by Name	232
8.5.2	Structural Equivalence	232
8.6	Compatibility and Conversion	234
8.7	Polymorphism	238
8.7.1	Overloading	239
8.7.2	Universal Parametric Polymorphism	240
8.7.3	Subtype Universal Polymorphism	242
8.7.4	Remarks on the Implementation	243
8.8	Type Checking and Inference	245
8.9	Safety: An Assessment	247
8.10	Avoiding Dangling References	248
8.10.1	Tombstone	248
8.10.2	Locks and Keys	250
8.11	Garbage Collection	251
8.11.1	Reference Counting	252
8.11.2	Mark and Sweep	254
8.11.3	Interlude: Pointer Reversal	255
8.11.4	Mark and Compact	256
8.11.5	Copy	257
8.12	Summary	260
8.13	Bibliographical Notes	260
8.14	Exercises	261
	References	264
9	Data Abstraction	267
9.1	Abstract Data Types	268
9.2	Information Hiding	270
9.2.1	Representation Independence	273
9.3	Modules	274
9.4	Summary	276
9.5	Bibliographical Notes	276

9.6	Exercises	278
	References	278
10	Object-Oriented Paradigm	279
10.1	The Limits of Abstract Data Types	280
10.1.1	A First Review	283
10.2	Fundamental Concepts	283
10.2.1	Objects	284
10.2.2	Classes	286
10.2.3	Encapsulation	289
10.2.4	Subtypes	289
10.2.5	Inheritance	295
10.2.6	Dynamic Method Lookup	300
10.3	Implementation Aspects	305
10.3.1	Single Inheritance	306
10.3.2	The Problem of Fragile Base Class	308
10.3.3	Dynamic Method Dispatch in the JVM	309
10.3.4	Multiple Inheritance	312
10.4	Polymorphism and Generics	318
10.4.1	Subtype Polymorphism	318
10.4.2	Generics in Java	320
10.4.3	Implementation of Generics in Java	325
10.4.4	Generics, Arrays and Subtype Hierarchy	325
10.4.5	Covariant and Contravariant Overriding	327
10.5	Summary	330
10.6	Bibliographical Notes	331
10.7	Exercises	331
	References	334
11	Functional Programming Paradigm	335
11.1	Computing Without State	335
11.1.1	Expressions and Functions	336
11.1.2	Computation as Reduction	338
11.1.3	The Fundamental Ingredients	340
11.2	Evaluation	341
11.2.1	Values	341
11.2.2	Capture-Free Substitution	342
11.2.3	Evaluation Strategies	342
11.2.4	Comparison of the Strategies	345
11.3	Programming in a Functional Language	347
11.3.1	Local Environment	347
11.3.2	Interactiveness	347
11.3.3	Pattern Matching	348
11.3.4	Programming in a Functional Style	349
11.3.5	Types	350
11.3.6	Infinite Objects	352

11.3.7	Imperative Aspects	353
11.4	An Assessment	355
11.5	Implementation: The SECD Machine	356
11.6	Fundamentals: The λ -Calculus	359
11.7	Summary	365
11.8	Bibliographical Note	366
11.9	Exercises	366
	References	367
12	Logic Programming Paradigm	369
12.1	Deduction as Computation	369
12.1.1	An Example	370
12.2	Syntax	373
12.2.1	The Language of First-Order Logic	373
12.2.2	Logic Programs	375
12.3	Theory of Unification	377
12.3.1	The Logic Variable	377
12.3.2	Substitution	378
12.3.3	Most General Unifier	381
12.3.4	A Unification Algorithm	383
12.4	The Computational Model	386
12.4.1	The Herbrand Universe	386
12.4.2	Declarative and Procedural Interpretation	387
12.4.3	Procedure Calls	388
12.4.4	Control: Non-determinism	392
12.4.5	Some Examples	394
12.5	Extensions	397
12.5.1	Prolog	397
12.5.2	Logic Programming and Databases	402
12.6	Advantages and Disadvantages of the Logic Paradigm	403
12.7	Summary	404
12.8	Bibliographical Notes	405
12.9	Exercises	406
	References	407
13	Constraint Programming Paradigm	409
13.1	Constraint Programming	409
13.1.1	Types of Problems	411
13.2	Constraint Logic Programs	413
13.2.1	Syntax and Semantics of CLP	414
13.3	Generate and Test Versus Constraint and Generate	417
13.3.1	A Further Example	420
13.4	MiniZinc	422
13.4.1	A MiniZinc CSP Model	422
13.4.2	A MiniZinc COP Model	425
13.5	Summary	427

13.6	Bibliographical Notes	427
13.7	Exercises	428
	References	431
14	Concurrent Programming	433
14.1	Threads and Processes	433
14.2	A Brief Historical Overview	434
14.3	Types of Concurrent Programming	436
14.3.1	Communication Mechanisms	438
14.3.2	Synchronisation Mechanisms	440
14.4	Shared Memory	441
14.4.1	Busy Waiting	442
14.4.2	Scheduler-Based Synchronisation	445
14.5	Message Exchange	451
14.5.1	Naming Mechanisms	451
14.5.2	Asynchronous Communication	453
14.5.3	Synchronous Communication	456
14.5.4	Remote Procedure Call and Rendez-Vous	457
14.6	Non-determinism and Parallel Composition	459
14.6.1	Parallel Composition	462
14.7	Concurrency in Java	463
14.7.1	Creation of Threads	463
14.7.2	Scheduling and Termination of Threads	464
14.7.3	Synchronisation and Communication Between Threads	466
14.8	Summary	469
14.9	Bibliographical Notes	469
14.10	Exercises	470
	References	471
15	Service-Oriented Programming Paradigm	473
15.1	Towards Services	473
15.1.1	Distributed Programs	473
15.1.2	Open Systems	474
15.1.3	Loose Coupling and Interoperability	475
15.1.4	Approaching Services	476
15.2	Elements of Service-Oriented Programming	477
15.2.1	Services, a Layered View	477
15.2.2	Data Types	481
15.2.3	Messaging Patterns	487
15.2.4	Operations and Interfaces	489
15.2.5	Behaviour	494
15.3	Service-Oriented Architectures	509
15.3.1	The First Generation: Web Services	510
15.3.2	Second Generation: Microservices and Serverless	513
15.4	Summary	515

15.5	Bibliographical Note	515
15.6	Exercises	516
	References	517
16	Short Historical Perspective	519
16.1	Beginnings	519
16.2	Factors in the Development of Languages	522
16.3	1950s and 1960s	523
16.4	1970s	527
16.5	1980s	532
16.6	1990s	536
16.7	2000s	542
16.8	2010s	544
16.9	Summary	547
16.10	Bibliographical Notes	548
	References	548
Index		551